

ISRM v1.0 — Reference Implementation

Validation Report

Validation run: 3 October 2026 • Originating specification: Marcel Reitz

1. Executive result

Three executable reference implementations were constructed against the ISRM v1.0 conformance matrix. Across 69 applicable implementation/test combinations, the run produced 69 PASS, 0 FAIL and 0 ERROR. Thirty-eight combinations were declared not applicable before execution because the reference implementation intentionally does not expose that capability.

DOI: <https://doi.org/10.5281/zenodo.23115842>

Public release: 3 October 2026 | Version 1.0 | License: CC BY 4.0

Originator: Marcel Reitz — Altenbuch, Germany

This result demonstrates internal implementability of the tested ISRM semantics. It is not independent certification, does not prove completeness, and does not establish novelty or formal standards compliance.

Implementation	PASS	FAIL	ERROR	N/A
deterministic_controller	13	0	0	17
tool_agent	27	0	0	3
recursive_multi_agent	29	0	0	1

2. Reference implementations

RI-1 Deterministic controller

A deliberately non-ML threshold/control architecture. Purpose: verify that ISRM does not secretly require an LLM, planner or learned model. Applicable tests emphasize event entry, uncertainty, execution state, feedback, trust, stale decisions and system boundary.

RI-2 Tool-using software agent

A stateful agent abstraction with explicit Candidate Action, authorization, bounded execution, identity/context revalidation, execution evidence and adaptation gates. Purpose: exercise the main intent-to-action semantics.

RI-3 Recursive multi-agent system

A coordinator/child-agent abstraction supporting delegation and nested delegation. Purpose: test recursive composition and the invariant that delegated authority cannot increase merely through recursion.

3. Additional property stress test

A separate deterministic randomized property check executed 10,000 generated cases over three core invariants: (a) child authority is a subset of delegated authority, (b) material uncertainty is not silently collapsed to certainty, and (c) a Candidate Action does not become authorized without an explicit authorization condition. Result: 0 invariant violations in 10,000 generated cases. This is a property stress test of the test harness semantics, not a mathematical proof of ISRM.

4. Detailed conformance results

System	Test	Result	Note
deterministic_controller	CT-01	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-02	PASS	
deterministic_controller	CT-03	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-04	PASS	
deterministic_controller	CT-05	PASS	
deterministic_controller	CT-06	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-07	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-08	PASS	
deterministic_controller	CT-09	PASS	
deterministic_controller	CT-10	PASS	
deterministic_controller	CT-11	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-12	PASS	
deterministic_controller	CT-13	N/A	Not applicable to declared reference implementation

deterministic_controller	CT-14	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-15	PASS	
deterministic_controller	CT-16	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-17	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-18	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-19	PASS	
deterministic_controller	CT-20	PASS	
deterministic_controller	CT-21	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-22	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-23	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-24	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-25	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-26	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-27	PASS	
deterministic_controller	CT-28	N/A	Not applicable to declared reference implementation
deterministic_controller	CT-29	PASS	
deterministic_controller	CT-30	PASS	
tool_agent	CT-01	PASS	
tool_agent	CT-02	PASS	
tool_agent	CT-03	PASS	
tool_agent	CT-04	PASS	

tool_agent	CT-05	PASS	
tool_agent	CT-06	PASS	
tool_agent	CT-07	PASS	
tool_agent	CT-08	PASS	
tool_agent	CT-09	PASS	
tool_agent	CT-10	PASS	
tool_agent	CT-11	PASS	
tool_agent	CT-12	PASS	
tool_agent	CT-13	N/A	Not applicable to declared reference implementation
tool_agent	CT-14	N/A	Not applicable to declared reference implementation
tool_agent	CT-15	PASS	
tool_agent	CT-16	PASS	
tool_agent	CT-17	PASS	
tool_agent	CT-18	PASS	
tool_agent	CT-19	PASS	
tool_agent	CT-20	PASS	
tool_agent	CT-21	PASS	
tool_agent	CT-22	PASS	
tool_agent	CT-23	PASS	
tool_agent	CT-24	N/A	Not applicable to declared reference implementation
tool_agent	CT-25	PASS	
tool_agent	CT-26	PASS	
tool_agent	CT-27	PASS	
tool_agent	CT-28	PASS	
tool_agent	CT-29	PASS	
tool_agent	CT-30	PASS	
recursive_multi_agent	CT-01	PASS	
recursive_multi_agent	CT-02	PASS	
recursive_multi_agent	CT-03	PASS	
recursive_multi_agent	CT-04	PASS	
recursive_multi_agent	CT-05	PASS	
recursive_multi_agent	CT-06	PASS	
recursive_multi_agent	CT-07	PASS	
recursive_multi_agent	CT-08	PASS	
recursive_multi_agent	CT-09	PASS	
recursive_multi_agent	CT-10	PASS	
recursive_multi_agent	CT-11	PASS	
recursive_multi_agent	CT-12	PASS	
recursive_multi_agent	CT-13	PASS	
recursive_multi_agent	CT-14	PASS	
recursive_multi_agent	CT-15	PASS	
recursive_multi_agent	CT-16	PASS	
recursive_multi_agent	CT-17	PASS	

recursive_multi_agent	CT-18	PASS	
recursive_multi_agent	CT-19	PASS	
recursive_multi_agent	CT-20	PASS	
recursive_multi_agent	CT-21	PASS	
recursive_multi_agent	CT-22	PASS	
recursive_multi_agent	CT-23	PASS	
recursive_multi_agent	CT-24	N/A	Not applicable to declared reference implementation
recursive_multi_agent	CT-25	PASS	
recursive_multi_agent	CT-26	PASS	
recursive_multi_agent	CT-27	PASS	
recursive_multi_agent	CT-28	PASS	
recursive_multi_agent	CT-29	PASS	
recursive_multi_agent	CT-30	PASS	

5. Interpretation and limitations

- The implementations were created from the same project that authored the candidate specification; therefore this is first-party validation, not independent validation.
- The harness validates semantic invariants and trace behavior. It does not yet connect to production APIs, PLC hardware, robots or safety-certified systems.
- N/A is not a pass. It means the capability is outside that reference implementation's declared scope.
- The all-pass result should trigger stronger adversarial and independent testing rather than confidence inflation.
- Before a formal standards proposal is presented as mature evidence, at least three external implementers should reproduce mappings and report ambiguities without being coached by the originator.

6. Prior-art implication

NIST 4D/RCS already demonstrates a recursive, hierarchical intelligent-systems reference architecture with sensory processing, world modeling, value judgment, behavior generation and closed feedback loops. Consequently, ISRM should not claim novelty for layered intelligent control, recursion or feedback as such. Its defensible proposal is the specific functional split and normative invariants around intent, context, decision/authorization, execution evidence, cross-layer control/trust/adaptation and conformance classes.

7. Next validation gate

1. Independent RI-A: industrial automation/PLC mapping by a third party.
2. Independent RI-B: tool-using AI agent implementation by a third party.
3. Independent RI-C: recursive/multi-agent implementation by a third party.
4. Record every disputed mapping and change the model when a dispute exposes a genuine ambiguity.
5. Repeat the 30-test suite after any normative change and version the results.